ML-DLOps Assignment-3

Fine-Tuning DistilBERT for Genre Classification

Siddhartha Singh
Roll No: **B22BB041**

Huggingface Model Link — GitHub Repository

---

1 Overview

This report presents a complete ML pipeline for **multi-class genre classification** of Goodreads book reviews into 8 genres. The workflow spans data acquisition, baseline modelling, fine-tuning a transformer model, containerization with Docker, and artifact publication to HuggingFace Hub and GitHub. The instructor-provided notebook was converted into modular production scripts (`utils.py`, `data.py`, `train.py`, `eval.py`) and packaged in Docker images for reproducible training and evaluation.

2 Project Structure

Component	**Description**
`src/utils.py`	Configuration constants (hyperparameters, paths, genre URLs), seed setting, device selection
`src/data.py`	Streaming download of Goodreads review data, train/test splitting, tokenization, `GenreDataset` class
`src/train.py`	TF-IDF + Logistic Regression baseline, DistilBERT fine-tuning via Trainer API, model saving
`src/eval.py`	Full evaluation (local and from HuggingFace repo), metric comparison, result serialization
`Dockerfile`	Training Docker image (runs `train.py`)
`Dockerfile.eval`	Production evaluation-only image (pulls model from HF Hub, runs `eval.py`)
`requirements.txt`	Python dependencies

Table 1: Project layout.

3 Model Selection

Model: `distilbert-base-cased` (65,787,656 parameters)

3.1 Rationale

- Retains ∼97% of BERT-base accuracy while being **60% faster** and **40% smaller** (65.8 M parameters vs. 110 M for BERT-base).
- Ideal for fine-tuning on limited compute; fits comfortably within a Docker container running on a single GPU.
- The **cased** variant preserves capitalisation, which carries genre-relevant signals in book reviews (proper nouns, titles, stylistic capitalisation in poetry/romance).
- Already provided notebook model is chosen.

The model and its fast tokenizer were loaded from the HuggingFace Hub inside the container:

```
from transformers import (DistilBertTokenizerFast,
                          DistilBertForSequenceClassification)
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-cased")
model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-cased", num_labels=8,
    id2label=id2label, label2id=label2id
).to(DEVICE)
```

4 Docker Environment

Two Dockerfiles were prepared, both based on `python:3.10-slim-bookworm`:

4.1 Training Image (`Dockerfile`)

```
FROM python:3.10-slim-bookworm
ENV PYTHONDONTWRITEBYTECODE=1  PYTHONUNBUFFERED=1
RUN apt-get update && apt-get install -y --no-install-recommends gcc g++ \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
CMD ["python", "src/train.py"]
```

```
docker build -t ml-dlops-train .
docker run --gpus all ml-dlops-train
```

4.2 Evaluation-Only Image (`Dockerfile.eval`)

```
FROM python:3.10-slim-bookworm
ARG HF_REPO
ENV HF_REPO=${HF_REPO}
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
CMD ["sh", "-c", "cd src && python eval.py --hf_repo ${HF_REPO}"]
```

```
docker build -f Dockerfile.eval \
  --build-arg HF_REPO=Sids27/distilbert-reviews-genres \
  -t ml-dlops-eval .
docker run --gpus all ml-dlops-eval
```

Key dependencies: `transformers`, `torch`, `scikit-learn`, `accelerate`, `huggingface_hub`, `pandas`, `seaborn`, `matplotlib`.

5 Training Summary

5.1 Dataset

Reviews were streamed from the UCSD Goodreads dataset (8 genre-specific `.json.gz` files) inside the training container. Per genre: first 10,000 reviews streamed, 2,000 randomly sampled, then 800 used for training and 200 for testing (from a 1,000-sample pool).

Split	**Samples**
Training	6,400
Testing	1,600
Genres	8

Table 2: Dataset split summary.

The 8 genres: `children`, `comics_graphic`, `fantasy_paranormal`, `history_biography`, `mystery_thriller_`
`poetry`, `romance`, `young_adult`.

5.2 Hyperparameters

Hyperparameter	**Value**
Epochs	3
Learning Rate	5×10^{-5}
Train Batch Size	10
Eval Batch Size	16
Warmup Steps	100
Weight Decay	0.01
Max Token Length	512
Optimizer	AdamW (default)
Evaluation Strategy	Every 100 steps
Save Strategy	Every epoch

Table 3: Training hyperparameters.

5.3 Training Output

The following metrics were produced by the training container (`docker run ml-dlops-train`):

Metric	**Value**
Training Loss	0.9908
Training Runtime	1608.37 s (≈26.8 min)
Samples / Second	11.94
Steps / Second	1.19
Total FLOPs	2.54×10^{15}

Table 4: Training metrics (container output).

After training, the model and tokenizer were pushed to the HuggingFace Hub at:

`https://huggingface.co/Sids27/distilbert-reviews-genres`

Artifacts uploaded: `model.safetensors` (263 MB), `config.json`, `tokenizer.json`, `tokenizer_config.js` `vocab.txt`, `label_mappings.json`. The model is **publicly accessible**.

6 Evaluation Comparison

Evaluation was performed by running the evaluation container (`docker run ml-dlops-eval`), which downloads the model from HuggingFace, streams the test data, and produces the results below.

6.1 Baseline: TF-IDF + Logistic Regression

Genre	Precision	Recall	F1-Score	Support
children	0.65	0.65	0.65	200
comics_graphic	0.79	0.70	0.75	200
fantasy_paranormal	0.34	0.30	0.32	200
history_biography	0.47	0.47	0.47	200
mystery_thriller_crime	0.54	0.52	0.53	200
poetry	0.62	0.74	0.67	200
romance	0.54	0.56	0.55	200
young_adult	0.37	0.39	0.38	200
Accuracy			**0.54**	1600
Weighted Avg	0.54	0.54	0.54	1600

Table 5: Baseline classification report (TF-IDF + Logistic Regression).

6.2 Fine-Tuned DistilBERT

Genre	Precision	Recall	F1-Score	Support
children	0.62	0.69	0.65	200
comics_graphic	0.84	0.77	0.80	200
fantasy_paranormal	0.40	0.41	0.41	200
history_biography	0.58	0.54	0.56	200
mystery_thriller_crime	0.60	0.61	0.60	200
poetry	0.76	0.79	0.77	200
romance	0.62	0.53	0.57	200
young_adult	0.39	0.45	0.42	200
Accuracy			**0.60**	1600
Weighted Avg	0.60	0.60	0.60	1600

Table 6: DistilBERT classification report (from evaluation container).

6.3 Local vs. HuggingFace Model Comparison

To verify serialization integrity, the model was evaluated both from the locally saved checkpoint and after re-downloading from the HuggingFace Hub:

Metric	Local Model	HuggingFace Model
Accuracy	0.5956	0.5956
Precision	0.6017	0.6017
Recall	0.5956	0.5956
F1	0.5974	0.5974
Eval Loss	1.2769	1.2769

Table 7: Local vs. HuggingFace model evaluation — metrics are identical.

6.4 Aggregate: Baseline vs. DistilBERT

Metric	Baseline (LR)	DistilBERT
Accuracy	0.5400	**0.5956**
Precision	0.5400	**0.6017**
Recall	0.5400	**0.5956**
F1	0.5400	**0.5974**
Eval Loss	–	1.2769

Table 8: Baseline vs. DistilBERT aggregate metrics.

DistilBERT yields an absolute improvement of **+5.56 percentage points** in accuracy over the TF-IDF + Logistic Regression baseline. The identical metrics between the local checkpoint and the HuggingFace-hosted model confirm that the SafeTensors serialization pipeline is lossless.

7 Challenges

1. **Long Docker Image Build Time:** Installing PyTorch and the full `transformers` stack inside a slim Python base image is time-consuming. The `pip install` layer alone takes over 15 minutes due to the large size of `torch` (>2 GB) and compilation of native extensions for packages like `tokenizers`. Multi-stage builds and layer caching were used to mitigate repeated rebuilds, but the initial build remains a bottleneck on machines without pre-cached layers.

2. **Large Dataset Streaming:** Each genre file is >100 MB compressed. Direct download would exhaust container memory, so streaming with early truncation (`HEAD=10,000`) and random sampling (`SAMPLE_SIZE=2,000`) was used to keep the data footprint manageable.

3. **Genre Ambiguity:** Several genres are inherently difficult to distinguish from review text alone (e.g., `fantasy_paranormal` vs. `young_adult`, both with F1 $\approx$ 0.41–0.42). This is a fundamental data-level challenge — reviews for these genres share overlapping vocabulary and themes.

4. **Reproducibility Across Environments:** Ensuring identical results between the locally saved model and the HuggingFace-hosted model required careful serialization (SafeTensors format) and matching tokenizer configurations. Table 6 confirms exact reproducibility.

5. **Docker GPU Access:** Running GPU workloads inside Docker requires the NVIDIA Container Toolkit. The Dockerfiles are designed to also work on CPU-only environments via automatic device detection in `utils.py`, though training on CPU is significantly slower.

8 Conclusion

- A TF-IDF + Logistic Regression **baseline** achieved 54% accuracy on 8-class genre classification.
- Fine-tuning **DistilBERT-base-cased** for 3 epochs improved accuracy to **59.56%** (+5.56 pp), with the strongest per-class performance on `comics_graphic` (F1 = 0.80) and `poetry` (F1 = 0.77).
- The model was pushed to HuggingFace Hub, and re-evaluation from the Hub confirmed **identical metrics**, validating the serialization pipeline.
- Docker images for both training and production evaluation were built and documented, with all code and artifacts available on GitHub.